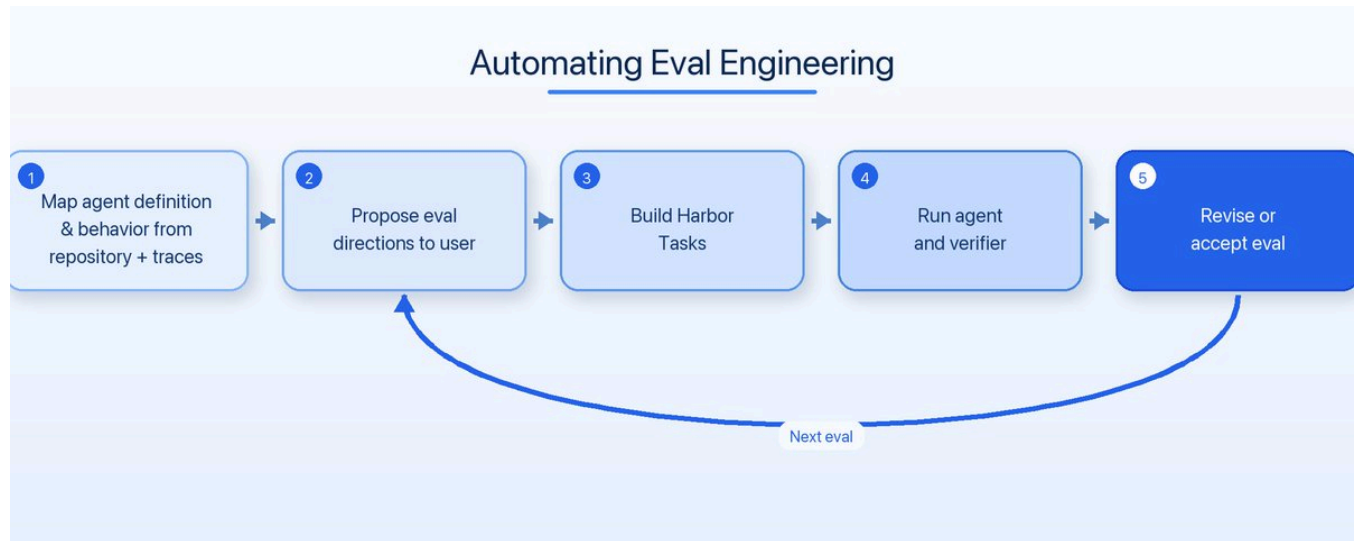


Towards Automating Eval Engineering

Viv (@Vtrivedy10) · July 22, 2026



Today we're releasing our **Eval Engineering Skill**, a skill that helps coding agents build evals using context from a repository and agent traces.

The skill inspects how an agent is structured, mines patterns from traces if available, and proposes abilities to test.

The skill is designed to interview the user who can give feedback on proposals and iteratively approve each eval. The end product is a set of executable evals in Harbor format.

Building the Environment & Task

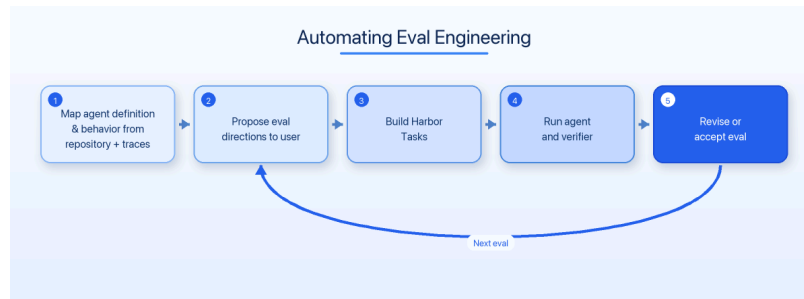
The skill first reads the repository and maps the agent surface including prompts, models, tools, skills, hooks, etc. It also identifies the data and services that back those behaviors such such as API calls.

Users can also point the agent to traces which can be retrieved using tools like the `langsmith-cli`. Traces show how tools behave in practice such as their arguments, results, and errors. These observed contracts help the skill reproduce relevant production behavior in a controlled environment.

Crawling the repo and traces gives the agent knowledge of a which abilities are important for the agent as it proposes eval tasks. We found that **interviewing the user**, leads to much better eval acceptance than one-shot generation. The user chooses from the proposed eval directions, and gives guidance on questions such as which tools & dependencies should run live or need to be simulated. For example, tool calls that incur

costs or require writes to production can be simulated instead of being run on every eval invocation.

We tested this flow on our documentation Q&A agent, chat-langchain. For this agent, the environment required a data corpus exposed through agent search tools modeled on the production agent. The tasks included realistic documentation question pulled from real traces and a verifier that checked the answer using a golden answer string and cited documents.



Eval Design is iterative

We found that while agents are sometimes able to one-shot evals, the best evals came from users providing feedback and specifying which capabilities were worth measuring in agents. Coding agents & skills provide a natural interface where domain knowledge on how to build a good eval are encoded and users can iterate over them over time.

For example, we found that when building verifiers, the first verifier was rarely the final one. A useful way to improve it was to run the eval and inspect both sides of the result:

- the agent trajectory, including its messages, tool calls, and actions.
- the verifier trajectory, evidence, reasoning, and final score.

This helped reveal if the task or verifier design was measuring what we cared about or if it could be **reward hacked** where agents could take shortcuts. These shortcuts could include overciting irrelevant sources to receive full credit on the eval, claim an action it never took, exploit exposed answer material, or satisfy a proxy without completing the task. Observing the traces for how agents solve problems often reveal the source of these failures. The task, environment, and verifier can then be revised and run again.

Evals are in Harbor format

The skill builds evals as Harbor tasks:

1. **An Instruction:** the message given to the agent at start describing the task
2. **An environment:** given as a Dockerfile containing the setup for the task such as what tools to install or what data to populate in the filesystem
3. **A verifier** that scores whether the agent completed the task correctly.

The skill builds these components together as a Harbor task:

Harbor runs the agent in the environment and records its trajectory, artifacts, reward, and errors. The same eval can then run against different models, prompts, tools, and agent versions.

Why this matters

Continual learning can be thought of as a continuous data mining problem where production data is used to build evals that improve agents over time. Teams mine traces to find recurring user requests, errors, failed tool calls, and incorrect state changes. which become evals so the same behavior can be measured and prevented in the future.

Evals are training data for agents. Teams can fit agent behavior to them through harness engineering such as changing prompts & tools or fine-tuning. The eval provides a fixed target for deciding whether those changes improved the intended capability.

Containerized evals make this process faster. The task and environment remain stable while the agent configuration changes, so builders can swap models, tools, prompts, or complete agent versions and compare results directly. Multiple configurations can run in parallel.

Reproducible environments are critical to that signal. When an eval mirrors the relevant tools, data, permissions, state, and failure modes from production, builders get a stable testbed that is still representative of how the agent operates. They can experiment quickly without relying on changing production systems or writing to production state.

The resulting loop is:

mine traces -> identify a failure -> build an eval -> improve the agent -> rerun

Try it today

The Eval Engineering Skill is available in the [langchain-ai/langchain-skills](#) repository.

Install the skill in Codex or Claude Code, open the repository containing the agent you want to evaluate, point to agent to a set of traces if available, and start with a simple prompt:

We're looking forward to expanding this skill and building tooling to make it easier to automatically build evals and fit agents to them autonomously.